SGNIA COMPANY · SOFTWARE INDUSTRIAL DE PRECISIÓN

Arquitectura Técnica y Estructura del Sistema

"Software que Protege y Hace Crecer tu Negocio"

MANUAL DE INGENIERÍA | OFICIAL 2026 | ANDROID SDK 36 | BLINDAJE ACTIVO

Gamaliel Velázquez Galicia · +52 276 109 5688 · sgnia.xyz

🏛️ Arquitectura Técnica y Estructura del Sistema SGNIA

SGNIA CARNICERÍA & BALANZA INDUSTRIAL POS

Desarrollado por SGNIA COMPANY — "Software que Protege y Hace Crecer tu Negocio"

---

1. Filosofía de Ingeniería y Principios Rectores

El sistema fue diseñado desde sus cimientos para operar en **entornos de misión crítica**: carnicerías industriales, rastros municipales, frigoríficos y puntos de venta mayoristas y minoristas.

1. **Filosofía Offline-First Absoluta**:

- Las operaciones de pesaje, caja, despiece, catálogo y validación de licencias **nunca dependen de internet ni de servidores externos**.
- Las básculas se comunican directamente a través de la red local Wi-Fi / Ethernet mediante sockets TCP/IP crudos.
- La persistencia de datos reside en la tablet mediante SQLite nativo con Room.

1. **Cero Dependencias Externas en Red**:

- Sin intermediarios MQTT, sin brokers en la nube, sin websockets de terceros. La comunicación con balanzas y hardware es directa a nivel de socket de capa de transporte.

1. **Sistema de Diseño Noir & Crema Bento Grid**:

- Interfaz de usuario diseñada como instrumental de precisión para cuartos fríos y ambientes con iluminación intensa o deficiente.
- Paleta con base en Carbón Mate (`#0F0E0D`, `#181715`, `#22201D`), acentos Crema Marfil (`#EEDDC8`) y Verde Esmeralda (`#4ADE80`) para estabilidad de pesaje.

---

2. Estructura Modular y Separación de Aplicaciones

El proyecto está organizado en módulos y variantes de producto (*Product Flavors*) que permiten compilar aplicaciones especializadas a partir de una base de código robusta:

```
Inventario/
├── app/                              # Módulo principal del sistema cárnico
│   ├── src/
│   │   ├── main/                     # Lógica compartida, repositorios, Room DB, red
│   │   ├── tienda/                   # SGNIA Carnicería POS (Tablet de Mostrador)
│   │   └── dueno/                    # SGNIA Consola Dueño (Dispositivo del Propietario)
├── keygen/                           # Módulo independiente: SGNIA Keygen Admin
│   └── src/main/                     # Panel del Desarrollador (Gamaliel Velazquez)
└── docs/                             # Documentación técnica y manuales operativos
```

Tabla de Aplicaciones Generadas

Aplicación	ID de Paquete	Flavour / Módulo	Destinatario	Funcionalidades Principales
SGNIA Carnicería POS	`com.example.inventariocarniceria.pos`	`:app` (tienda)	Tablet de Mostrador	Visor multiescala dinámico (1 a 10+ básculas), tara independiente, recepción de mercancía con pesaje y códigos de barras, despiece y mermas, pedidos de surtido y activación de licencia por Hardware ID.
SGNIA Consola Dueño	`com.example.inventariocarniceria.admin`	`:app` (dueno)	Teléfono del Dueño	Unificación Maestra multi-tienda (consolida pedidos de todas las sucursales para el rastro con WhatsApp y PDF Oficial), supervisión de sucursales con PINs y directorio de proveedores.
SGNIA Keygen Admin	`com.example.sgniakeygen`	`:keygen`	Celular de Gamaliel	Gestión de cartera de clientes agrupados por Dueño y Sucursales, generador criptográfico HMAC de llaves (10m, 7d, 1m, 1y, LIFE), tarifario maestro, control de pagos y emisión de comprobantes oficiales SGNIA.

3. Capa de Datos y Persistencia

Base de Datos Local (Room SQLite)

Gestionada en `app/src/main/java/com/example/inventariocarniceria/data/local/`:

- `InventoryDatabase.kt`: Base de datos SQLite con migraciones destructivas controladas para desarrollo ágil y versionado de entidades.
- **Entidades Principales**:
- `ProductEntity`: Códigos PLU, nombres de corte, precios por kilo, stock actual, mermas esperadas y categoría cárnica.

- `OrderEntity` & `OrderItemEntity`: Pedidos de tienda con cálculo automático de faltantes y redondeo a cajas cerradas.
- `BranchEntity`: Directorio de sucursales con encargados y PINs de mostrador.
- `SupplierEntity`: Proveedores y frigoríficos asociados.

Preferencias Seguras y Configuración de Hardware

- `Jetpack DataStore Preferences`: Almacena el consentimiento legal (`LegalAcceptanceScreen`), preferencias de tema y último estado operativo.
- `SharedPreferences (scales_json_list)`: Almacena el catálogo de básculas configuradas en formato JSON serializado (ID, Nombre, IP, Puerto, Marca, Modelo, Protocolo) garantizando persistencia inmediata ante reinicios del dispositivo.

---

4. Concurrencia y Motor de Red en Segundo Plano

La gestión de comunicación con básculas reside en `TcpScaleManager.kt`:

- **SupervisorJob con Coroutines de Kotlin**:
- Cada báscula opera en un hilo secundario independiente (`Dispatchers.IO`) dentro de un `ConcurrentHashMap<Int, Job>`.
- Si una báscula se apaga o pierde señal Wi-Fi, su worker de reconexión intenta enlazar cada 2 segundos sin congelar la interfaz de usuario ni afectar a las demás básculas conectadas.
- **Prevención de Colisión Mono-Cliente**:
- Muchos módulos de red en básculas (como Lantronix xPico) son mono-cliente (admiten un solo socket abierto simultáneamente).
- El sistema pausa temporalmente el hilo de lectura al realizar pruebas de ping o extracción de catálogos PLU y lo reactiva de inmediato en el bloque `finally`.

---

5. Instrucciones de Compilación y Verificación

El proyecto se compila y verifica con Gradle desde la terminal:

```
# 1. Compilación de verificación de tipos en Kotlin
.\gradlew.bat :app:compileTiendaDebugKotlin :app:compileDuenoDebugKotlin :keygen:compileDebugKotlin

# 2. Ejecución de la suite completa de pruebas unitarias (37 pruebas)
.\gradlew.bat testDebugUnitTest

# 3. Compilación de todas las variantes
.\gradlew.bat assembleTiendaDebug assembleDuenoDebug :keygen:assembleDebug
```

_{Documentación Técnica Oficial · SGNIA COMPANY © 2026}